

LAB ACTIVITY 10: Database Operations in JSP

Application	StudentRegistrationSystem
Database	student_registration
Server	GlassFish
Database server	Java DB (Derby Network Server)

Learning Outcomes

- Create and manage related Derby tables.
- Use JDBC to retrieve records from multiple tables.
- Apply SQL INNER JOIN.
- Implement database transactions with commit and rollback.

Task 1: Create and Connect the Database

Step 1 - Start services

From Services, start Java DB and GlassFish.

Step 2 - Create database

Right-click Java DB > Create Database. Use student_registration / app / app.

Step 3 - Open SQL editor

Connect to the database, right-click its Tables node, and select Execute Command.

Task 2: Create Related Tables

```
CREATE TABLE users (
  user_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(100) NOT NULL,
  full_name VARCHAR(100)
);

CREATE TABLE student (
  student_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL
);

CREATE TABLE course (
  course_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  semester INT NOT NULL
);

CREATE TABLE registration (
  registration_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_id INT NOT NULL,
```

```

course_id INT NOT NULL,
registration_date DATE NOT NULL,
CONSTRAINT fk_reg_student FOREIGN KEY (student_id) REFERENCES student(student_id),
CONSTRAINT fk_reg_course FOREIGN KEY (course_id) REFERENCES course(course_id)
);

```

Execution order: Create the parent tables STUDENT and COURSE before REGISTRATION because REGISTRATION contains foreign keys.

Task 3: Insert Sample Data

```

INSERT INTO users (username, password, full_name)
VALUES ('admin', 'admin123', 'System Administrator');
INSERT INTO users (username, password, full_name)
VALUES ('lecturer', 'lect123', 'Lecturer User');

```

```

INSERT INTO student (student_name, email)
VALUES ('Ali Ahmad', 'ali@example.com');
INSERT INTO student (student_name, email)
VALUES ('Siti Aminah', 'siti@example.com');

```

```

INSERT INTO course (course_name, semester)
VALUES ('Java Programming', 4);
INSERT INTO course (course_name, semester)
VALUES ('Database System', 4);

```

```

INSERT INTO registration (student_id, course_id, registration_date)
VALUES (1, 1, DATE('2026-05-20'));
INSERT INTO registration (student_id, course_id, registration_date)
VALUES (2, 2, DATE('2026-05-20'));

```

Users

#	USER_ID	USERNAME	PASSWORD	FULL_NAME
1	1	admin	admin123	System Administrator
2	2	lecturer	lect123	Lecturer User

Student

#	STUDENT_ID	STUDENT_NAME	EMAIL
1	1	Ali Ahmad	ali@example.com
2	2	Siti Aminah	siti@example.com

Course

#	COURSE_ID	COURSE_NAME	SEMESTER
1	1	Java Programming	4
2	2	Database System	4

Registration

#	REGISTRATION_ID	STUDENT_ID	COURSE_ID	REGISTRATION_DATE
1	1	1	1	1 2026-05-20
2	2	2	2	2 2026-05-20

Task 4: Create the GlassFish Project

Step 1 - File > New Project > Java Web > Web Application.

Step 2 - Set Project Name to StudentRegistrationSystem.

Step 3 - Select GlassFish Server and finish the wizard.

Step 4 - Add derbyclient.jar (or Java DB Client library) to the project libraries.

Task 5: Database Connection

Create WEB-INF/dbConnection.jsp:

```
<%@ page import="java.sql.*" %>
<%
Connection conn = null;
try {
    Class.forName("org.apache.derby.jdbc.ClientDriver");
    conn = DriverManager.getConnection(
        "jdbc:derby://localhost:1527/student_registration", "app", "app");
} catch (Exception e) {
    out.println("<p>Database connection error: " + e.getMessage() + "</p>");
}
}%>
```

Task 6: Display Data with INNER JOIN

Create displayData.jsp:

```
<%@ page import="java.sql.*" %>

<%@ include file="WEB-INF/dbConnection.jsp" %>

<!DOCTYPE html>

<html>
<head>
    <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Information</h2>

<table border="1">
<tr>
    <th>Student Name</th>
    <th>Course Name</th>
    <th>Semester</th>
    <th>Registration Date</th>
</tr>

<%
String sql = "SELECT s.student_name, c.course_name, c.semester, r.registration_date " +
    "FROM registration r " +
    "INNER JOIN student s ON r.student_id = s.student_id " +
    "INNER JOIN course c ON r.course_id = c.course_id " +
    "ORDER BY s.student_name";

PreparedStatement ps = null;
ResultSet rs = null;
```

```

if (conn != null) {

    try {

        ps = conn.prepareStatement(sql);

        rs = ps.executeQuery();

        while (rs.next()) {
%>

<tr>

    <td><%= rs.getString("student_name") %></td>

    <td><%= rs.getString("course_name") %></td>

    <td><%= rs.getInt("semester") %></td>

    <td><%= rs.getDate("registration_date") %></td>

</tr>

<%

    }

} catch (SQLException e) {

    out.println("<p>Query error: " + e.getMessage() + "</p>");

} finally {

    try {

        if (rs != null) {

            rs.close();

        }

        if (ps != null) {

            ps.close();

        }

        if (conn != null) {

            conn.close();

        }

    } catch (SQLException e) {

        // Ignore close error

    }

}

}

```

```
%>
```

```
</table>
```

```
</body>
```

```
</html>
```

Step 1 - Run

Start both servers, right-click displayData.jsp > Run File, and verify the joined registration data.

Student Registration Information

Student Name	Course Name	Semester	Registration Date
Ali Ahmad	Java Programming	4	2026-05-20
Siti Aminah	Database System	4	2026-05-20

Task 7: Implement a Transaction

Create transaction.jsp. This transaction inserts a student and a course. If either operation fails, both are rolled back.

```
<%@ page import="java.sql.*" %>
```

```
<%@ include file="WEB-INF/dbConnection.jsp" %>
```

```
<%
```

```
PreparedStatement ps1 = null;
```

```
PreparedStatement ps2 = null;
```

```
if (conn != null) {
```

```
try {
```

```
    conn.setAutoCommit(false);
```

```
    ps1 = conn.prepareStatement(
```

```
        "INSERT INTO student(student_name, email) VALUES (?, ?)";
```

```
    ps2 = conn.prepareStatement(
```

```
        "INSERT INTO course(course_name, semester) VALUES (?, ?)";
```

```
    ps1.setString(1, "Ahmad");
```

```
    ps1.setString(2, "ahmad@example.com");
```

```
    ps1.executeUpdate();
```

```

ps2.setString(1, "Web Programming");
ps2.setInt(2, 4);
ps2.executeUpdate();

conn.commit();

out.println("<h3>Transaction successful!</h3>");

} catch (SQLException e) {

    try {
        conn.rollback();
    } catch (SQLException rollbackError) {
        // Ignore rollback error
    }

    out.println("<h3>Transaction failed and was rolled back.</h3>");
    out.println("<p>" + e.getMessage() + "</p>");

} finally {

    try {
        if (ps1 != null) {
            ps1.close();
        }

        if (ps2 != null) {
            ps2.close();
        }

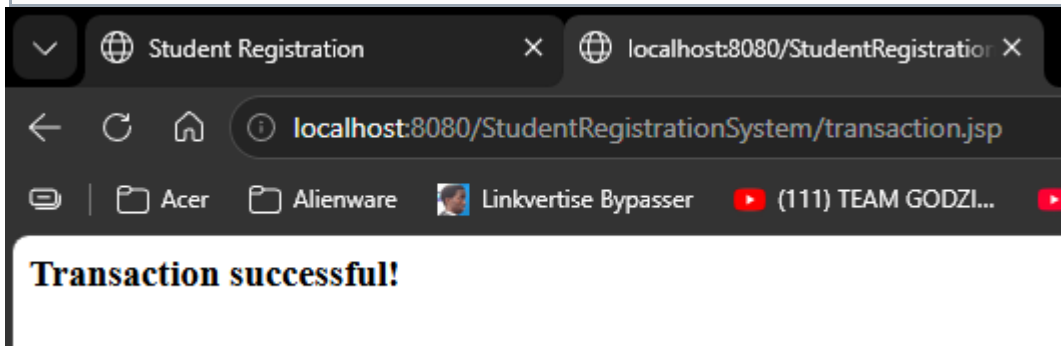
        if (conn != null) {
            conn.setAutoCommit(true);
            conn.close();
        }

    } catch (SQLException e) {
        // Ignore close error
    }
}
}
%>

```

ACTIVITY EVIDENCE

Paste the successful transaction output and SQL results proving both rows were inserted.



Exercise: Online Food Ordering System

Develop the system using JSP, JDBC, Java DB, GlassFish and NetBeans 8.2. Do not use XAMPP, phpMyAdmin or MySQL.

- Create Derby database food_ordering with users, customer, food and orders tables.
- Use Derby identity columns for all primary keys.
- Build staff login, insert order, display, update, delete and transaction modules.
- Use PreparedStatement for user input.
- Use INNER JOIN to display customer name, food name, price, quantity and total price. Use commit and rollback when processing an order and its related records.

Data modelling hint: ORDERS may be acceptable in Derby, but a clearer table name such as FOOD_ORDER avoids confusion. Store unit price in FOOD and calculate total price as price * quantity in the SELECT query.

Reflection

How do foreign keys and transactions protect the consistency of related records?

Foreign keys prevent invalid relationships (such as orphan records), while transactions enforce all-or-nothing atomicity via commit() and rollback() to prevent partial or corrupted data writes.

Conclusion

Students created related Derby tables, retrieved joined data, and implemented commit/rollback transactions in JSP on GlassFish.

Appendix A: Derby Troubleshooting

Symptom	Check / Action
Connection refused on localhost:1527	Start Java DB from Services > Databases > Java DB > Start Server.
ClassNotFoundException: org.apache.derby.jdbc.ClientDriver	Add derbyclient.jar or the Java DB Client library to the web project, then clean and build.

Database not found	Confirm the database name in the URL exactly matches the database created in Java DB.
Authentication error	Use the username and password entered when the database was created (this guide uses app/app).
Table does not exist	Derby normally stores unquoted identifiers in uppercase. Connect as APP and check APP.STUDENT or APP.REGISTRATION.
Port already in use	Stop the other Derby process or configure the Java DB server and JDBC URL to use the same alternative port.
GlassFish page fails but SQL editor works	Confirm GlassFish is running and derbyclient.jar is packaged with the project. Clean and redeploy.
Identity insert error	Do not supply a value for a GENERATED ALWAYS identity column; omit the ID column from INSERT.

Appendix B: Required Replacements from the MySQL Edition

Remove / Replace	Use instead
XAMPP Control Panel	NetBeans Services window
Apache HTTP Server in XAMPP	GlassFish Server
MySQL/MariaDB service	Java DB / Derby Network Server
phpMyAdmin	NetBeans SQL editor (Execute Command)
mysql-connector-j.jar	derbyclient.jar / Java DB Client library
com.mysql.cj.jdbc.Driver	org.apache.derby.jdbc.ClientDriver
jdbc:mysql://localhost:3306/database	jdbc:derby://localhost:1527/database
root / blank password	The Derby account created for the database; app / app in this guide
AUTO_INCREMENT	GENERATED ALWAYS AS IDENTITY
MySQL-specific setup and troubleshooting	Derby/GlassFish setup and troubleshooting

Submission Checklist

- ☐ GlassFish and Java DB start successfully.
- ☐ All requested databases and tables exist under the APP schema.
- ☐ The Derby client driver is included in each web project.

- ☐ JDBC URLs use port 1527 and the correct Derby database name.
- ☐ All JSP pages run without SQL or driver errors.
- ☐ PreparedStatement is used for user-supplied values.
- ☐ Connection, statement and result resources are closed.
- ☐ Screenshots are inserted in the marked evidence areas.